

Recurrence, Tilings & State Evolution - Student Pack

State before recurrence. Define the object/state, prove exactly-once transitions, give meaningful bases, then compute or choose a smaller representation.

Recurrence, Tilings & State Evolution

The common idea is remember exactly what the future needs. A recurrence is not a pattern guessed from a few numbers. It is a counting identity produced by a precise state and an exactly-once decomposition.

1. Reconnect: a symbol must name an object

Suppose T_n counts tilings of a $2 \times n$ rectangle by 2×1 dominoes. Before any recurrence is written, the object is fixed: one complete domino tiling of that rectangle.

Look at the left edge. Either one vertical domino is placed, leaving a $2 \times (n-1)$ rectangle, or two horizontal dominoes start together, leaving a $2 \times (n-2)$ rectangle. These cases are disjoint and exhaustive, so

$$T_n = T_{n-1} + T_{n-2}.$$

The base states have combinatorial meaning: $T_0=1$ because the empty board has one empty tiling, and $T_1=1$ because the 2×1 board has one tiling.

The order of thought is therefore:

OBJECT -> STATE -> EXACTLY-ONCE SPLIT -> SMALLER STATE -> BASE STATES -> RECURRENCE.

2. State sufficiency: can two histories with the same state have different futures?

A state is sufficient when it remembers every fact that can change future legal choices, and no irrelevant history.

Consider compositions of n using parts 1 and 2, with no two consecutive 2s. The remaining sum alone is not sufficient: arriving with previous part 1 and arriving with previous part 2 give different legal next moves. Add one memory bit: whether the previous part was 2.

A useful falsifier is:

Can two histories with this same proposed state have different legal futures?

If yes, the state is too small. If no coordinate affects future choices, the state may be too large.

3. One-state recurrence versus hidden memory

For binary strings with no consecutive 1s, a one-state recurrence can still work after a structural first-symbol split. Let A_n count valid strings of length n .

- first symbol 0: append any valid string of length $n-1$;
- first symbols 10: append any valid string of length $n-2$.

Thus $A_n = A_{n-1} + A_{n-2}$ with $A_0=1$, $A_1=2$.

But if the condition also requires an even number of 1s, the future acceptance condition depends on parity. A state must now remember both the last-bit restriction and the parity of the number of 1s used so far. The recurrence becomes a small transition table rather than one scalar formula.

The lesson is state first, not scalar recurrence always.

4. Tilings: freeze the first unresolved region

For a board or strip, identify the first unresolved cell or column. List all legal ways to cover it, and stop each branch as soon as the remaining region becomes a previously defined state.

Three checks are mandatory:

1. every complete tiling begins in one listed branch;
2. no complete tiling begins in two branches;
3. each branch leaves a smaller canonical state.

If a special tile can be used at most once, the board size alone may not determine the future. Add a flag recording whether the special tile has already been used.

5. Deterministic state graphs and reverse search

A process with legal moves creates a directed graph: states are vertices and legal moves are directed edges. Multiple choices do not make the problem a game unless an opponent is choosing moves to optimize an outcome.

For the machine $x \rightarrow x+1$ or $x \rightarrow 2x$, a forward search from 1 can branch rapidly. If the target is fixed, reverse predecessors are often simpler:

- predecessor $y-1$ always exists for the $+1$ move;
- predecessor $y/2$ exists when y is even for the doubling move.

Searching backward from the target explores the same reachability structure in the opposite direction. The direction is a representation choice, not a change in the mathematics.

6. Carry state: local memory can replace global enumeration

Suppose a number is represented as a sum of powers of 2, with each power usable at most twice. A global list of partitions is possible but quickly becomes messy. Process binary positions from low to high.

At each position, the legal choice depends on the target digit and the incoming carry. Therefore a local state such as

(digit position, incoming carry)

contains the information needed for future choices. The arithmetic rule producing the carry must already be understood; the new combinatorial work is to count the legal state transitions.

7. Recurrence is not always the best endpoint

State thinking may reveal a simpler non-recursive representation.

Example: words containing 4 E's and 3 N's with no consecutive N's. Place the four E's first. They create five gaps, and choose three of those gaps for the N's. The count is $C(5,3)=10$.

A state recurrence can count the same objects, but the gap representation is smaller. A strong solver asks for the cheapest correct representation rather than forcing recurrence.

8. Mandatory contrasts

Supplied algebraic recurrence vs counting recurrence

If a recurrence is given and the main work is algebraic manipulation, use recurrence algebra already learned. If the relation itself must be proved from what is counted, define the state and derive it structurally here.

Direct count vs recursive decomposition

A recurrence is natural when a first or last structural move leaves smaller copies of the same state. A direct count is better when a single representation, symmetry, or gap choice collapses the whole problem.

Deterministic evolution vs adversarial game

A deterministic reachability problem may have many legal moves, but there is no opponent. Winning/losing strategy belongs only when another player chooses against you.

One state vs hidden memory

If two histories with the same proposed state can have different futures, enrich the state. If they cannot, do not carry unnecessary history.

9. Historical source anchors

The validated IOQM anchors illustrate five different representations:

- 2023 Q08: tiling decomposition from the first unresolved region;
- 2024 Q20: deterministic reverse-state search;
- 2024 Q14: sparse near-boundary state history, where recurrence is not forced;
- 2023 Q21: residual/partition representation;
- 2023 Q26: carry-state representation counting.

Their verified answers are 59, 10, 80, 15, 19 respectively. Historical stems remain source-controlled; the lesson is the representation and first move, not memorization of a paper solution.

10. Adopt the router

For a new problem, write these questions before arithmetic:

1. What exactly is one object or reachable state?
2. What information must the state remember for future legality?
3. Can every object be assigned to exactly one first or last transition?
4. What smaller state remains in each branch?
5. What do the base states mean?
6. Is one state enough, or is hidden memory required?
7. Would reverse search, symmetry, gaps, residual structure, or a carry table be smaller than a recurrence?
8. Can small cases independently verify the model?

Define the state before writing the recurrence.

First-Step Reference: State Before Recurrence

Use this page before doing arithmetic.

A. Name the object or target

Write one sentence:

My state counts/records ...

A symbol without a precise meaning is not a model.

B. Test whether the state is sufficient

Ask:

Can two histories with this same proposed state have different legal futures?

- Yes -> add the missing memory.
- No -> the state may be sufficient.
- If some remembered detail never changes future choices -> remove it.

C. Try an exactly-once first/last split

For each branch check:

Does every valid object enter exactly one branch?

If branches overlap, do not add them. Redesign the split or use the appropriate counting method.

D. Map branches to smaller states

A recurrence term is justified only when the branch is in a one-to-one correspondence with the claimed smaller state, with any transition choices accounted for.

E. Give base-state meaning

Write what each initial value counts. Do not treat $T_0=1$ as a magic convention: explain the empty configuration when it is used.

F. Choose the representation

- board/tiling -> freeze the first unresolved region;
- string/path -> first or last symbol/step;
- hidden local restriction -> add a small flag/state;
- fixed target in a branching process -> compare reverse predecessors;
- digit representation -> process a digit with incoming carry;
- residual/near-boundary structure -> test a direct partition or deficit model;
- opponent optimizes choices -> this is a game, not deterministic state evolution.

G. Verify

Check the first few small cases independently. A recurrence matching a short numerical pattern is not proof; the structural decomposition is the proof.

Recognition Lab

Choose the most useful first representation or diagnosis. Do not solve the whole problem unless needed.

1. A $2 \times n$ board is tiled by dominoes. Which first move is most useful?
A. List complete tilings. B. Freeze the leftmost unresolved column. C. Guess a Fibonacci pattern. D. Use inclusion-exclusion.
2. Binary strings have no consecutive 1s and must contain an even number of 1s. What extra information must a local state remember besides position?
A. The entire prefix. B. Last bit and parity of the 1-count. C. Only the number of zeros. D. Nothing.
3. A machine permits $x \rightarrow x+1$ and $x \rightarrow 2x$; one fixed large target is given. What should be compared first?
A. Forward and reverse branching. B. AP and GP formulas. C. Inclusion-exclusion. D. Prime factorization only.
4. Four E's and three N's must be arranged with no adjacent N's. Which representation is smallest?
A. A recurrence table is mandatory. B. Choose three of the five gaps around the E's. C. A game tree. D. A geometric diagram.
5. A proposed recurrence uses two first-step cases, but one object satisfies both descriptions. What is wrong?

- A. Initial values are too small. B. Cases overlap, so naive addition double-counts. C. The state is too large. D. Reverse search is impossible.
6. Two prefixes have the same remaining length but different legal next symbols. What does this prove?
- A. Remaining length alone is an insufficient state. B. A recurrence cannot exist. C. The problem is adversarial. D. Complement counting is required.
7. A problem already supplies $a_{n+2}=3a_{n+1}-2a_n$ and asks for a high-index term. Which work is central?
- A. Derive a counting-state recurrence. B. Use the supplied recurrence algebra/sequence tools. C. Build a tiling. D. Search backward in a state graph.
8. A process offers several legal moves, but no opponent chooses against you. Which description is correct?
- A. It is automatically a two-player game. B. It is a deterministic state/reachability problem with branching choices. C. It must use minimax. D. It cannot be represented by a graph.

First-Line Lab

Write only the first structural line or state definition. The target is the model, not the final arithmetic.

1. A $2 \times n$ rectangle is tiled by dominoes. Define the counting state before writing any recurrence.
2. Count binary strings of length n with no consecutive 1s. Give an exactly-once first-symbol split.
3. Count compositions of n using 1 and 2 with no consecutive 2s. State the extra memory a recurrence needs.
4. Starting from 1, a machine allows $x \rightarrow x+1$ or $x \rightarrow 2x$. The target is 100. Write the reverse predecessor rule before searching.
5. A number is represented using powers of 2, each usable at most twice. State a local representation suitable for left-to-right or low-to-high counting.
6. Count partitions of 8 into distinct parts. Give a compact include/exclude state or a direct representation; do not force a one-variable recurrence.
7. Count length- n words over $\{A,B,C\}$ with adjacent letters different. State the symmetry that collapses the transition count.
8. A story describes repeated legal moves and asks whether a target can be reached. Write the question that decides whether this belongs to deterministic state evolution or adversarial-game strategy.

Practice & Transfer Bank

Attempt each item without a method label. The stages gradually remove support, but the governing habit is unchanged: define the state before the recurrence.

Stage 0

1. Let a_n count tilings of a $1 \times n$ strip by tiles of lengths 1 and 2. Derive the recurrence from the first tile and find a_6 .
2. Let b_n count binary strings of length n with no consecutive 1s. Derive a first-symbol recurrence and find b_5 .
3. Let T_n count domino tilings of a $2 \times n$ rectangle. Explain the two left-edge branches and find T_5 .
4. A staircase has 7 steps. Each move climbs 1 or 2 steps. Classify by the final move and count the routes.

Stage 1

5. A 1×7 strip is tiled by pieces of lengths 1 and 3. Derive the recurrence and count the tilings.
6. How many length-5 words over $\{A,B,C\}$ have adjacent letters different? Give both a state-transition explanation and the shorter symmetry count.
7. A $2 \times n$ board may be tiled by dominoes and by 2×2 square tiles. A width-2 block can therefore be filled in two structurally different ways. Derive the recurrence and find the count for $n=5$.
8. Count binary strings of length 7 with no consecutive 1s. Your proof must explain why the first-step branches are disjoint and exhaustive.

Stage 2

9. How many compositions of 8 using parts 1 and 2 contain no consecutive 2s? Use enough state to make future legality well-defined.
10. How many length-6 binary strings have no consecutive 1s and an even number of 1s? Use a state that remembers both local legality and acceptance parity.
11. How many length-6 binary strings contain no run of three equal bits? Use a finite-memory state or an equivalent recurrence whose state meaning is explicit.
12. How many length-6 words over $\{A,B,C\}$ have adjacent letters different and contain an even number of A's? Identify the memory that a transition table must retain.

Stage 3

13. Starting from 1, a move replaces x by $x+1$ or $2x$. Find the minimum number of moves needed to reach 31. Justify minimality, not only a path.
14. For the same machine, find the minimum number of moves needed to reach 100. Compare forward and reverse search before choosing a direction.
15. How many partitions of 8 into distinct positive parts are there? Use an include/exclude state or another compact representation and verify the result independently.
16. In how many ways can 10 be written as a sum of powers of 2 if each power may be used 0, 1, or 2 times? Use a bounded-part or carry-state representation.

Stage 4

17. A 1×10 strip is tiled by pieces of lengths 1 and 3. Count the tilings and state the base cases needed by your recurrence.
18. Count length-8 binary strings with no consecutive 1s and an even number of 1s. Do not list all 256 strings.
19. Starting from 2, a machine allows $x \rightarrow 2x$ and $x \rightarrow x+3$. Find the minimum number of moves to reach 29 and prove that fewer moves are impossible.
20. Arrange four E's and three N's with no consecutive N's. Count the arrangements. Then explain why a gap representation is cleaner here than a recurrence, even though a recurrence is possible.

Mixed Mastery Test

1. A 1×9 strip is tiled by pieces of lengths 1 and 2. How many tilings are possible? Give a structural justification.
2. How many binary strings of length 8 contain no consecutive 1s?
3. How many compositions of 9 using parts 1 and 2 contain no consecutive 2s? Your state must make future legality unambiguous.
4. A $2 \times n$ board is tiled by dominoes and 2×2 square tiles. How many tilings are there for $n=6$?
5. How many length-7 binary strings contain no consecutive 1s and have an even number of 1s?
6. Starting from 1, a move replaces x by $x+1$ or $2x$. What is the minimum number of moves needed to reach 63? Prove minimality.
7. How many partitions of 10 into distinct positive parts are there?
8. In how many ways can 18 be written as a sum of powers of 2 if each power may be used at most twice?
9. You are counting binary strings with no consecutive 1s and an even number of 1s. A proposed state records only the current position. Explain precisely why it is insufficient and give a sufficient small state.
10. A process allows several legal moves from a state and asks whether a target is reachable. State the additional feature that would turn this into an adversarial game problem rather than deterministic state evolution, and explain why branching alone is not enough.